

AsymptoticInverse after nine reviews

A delta audit with native Wolfram Language experiments

Prepared for Vladimir Reshetnikov

9 September 2026

Audited repository: [VladimirReshetnikov/Asymptotic](#)

Pinned commit: 921387e5ba1239bfda96e63e64e89bf63d9c41e6

Native execution environment: Wolfram Language 15.0.0, Linux x86-64

Standalone loaded: 574,894 bytes

Abstract. This report examines the current implementation against the repository's existing review register, rather than presenting its accumulated backlog as a new audit. Two additional observations survive that comparison: a relative-accuracy certificate request can stagnate because successful but insufficient attempts can repeat without increasing arithmetic precision, and `SeriesRefine` can coarsen its returned approximation. The first is a reproducible adaptation defect; the second is a concrete API-policy issue, not an invalid asymptotic formula.

Two previously recorded concerns receive materially stronger evidence. Native executions confirm that several public entry points accept provably nonreal constants while another rejects the same constant. Native executions also confirm the previously diagnosed nonlinear logarithmic-frontier defect in the current snapshot. For the latter, the archive supplies an independently tested closed-frontier implementation that computes the complete omitted boundary polynomial instead of relying only on a worst-case degree estimate.

Four paired experiments establish both overlap with built-ins and a narrower practical distinction. Polynomial reversion and a three-term Stirling expansion agree with native results; two particular generalized inverse calls remain unevaluated in `AsymptoticSolve` while the package returns explicit expansions. These are versioned observations, not claims of universal limitations in Wolfram Language.

The accompanying archive contains the article sources, evidence records, reproductions, desired-behavior tests, a Python prototype, and explicitly experimental certificate fixes. The repository-wide native test suite and the complete patch candidates were not validated in this audit.

Contents

1	Results and scope	3
1.1	Snapshot and reproducibility boundary	3
1.2	What the existing reviews remove from this report	4
1.3	Severity is about the contract, not just the output type	4
2	The implementation's mathematical and architectural contract	5
2.1	A positive local coordinate and a sparse coefficient algebra	5
2.2	Inverse construction and the role of retained evidence	5
2.3	The specialized engines are not one universal series ring	6
2.4	Boundaries checked without inventing additional bugs	6
3	Comparison with Wolfram Language built-ins	6
3.1	The documented boundary	6
3.2	Executed comparison B01: ordinary polynomial reversion	8
3.3	Executed comparison B02: a logarithmic inverse	8
3.4	Executed comparison B03: an irrational correction weight	8
3.5	Executed comparison B04: the Stirling carrier	9
3.6	What these experiments do not establish	9
4	N01: certificate accuracy can stagnate at fixed arithmetic precision	10
4.1	A small exact problem separates the failure modes	10
4.2	The first cause: relative tolerance does not size the arithmetic	11
4.3	The second cause: a successful proof is mistaken for sufficient arithmetic progress	11
4.4	Why this does not undermine the interval proof itself	11
4.5	A targeted repair and the supplied candidates	12
4.6	A better precision controller than unconditional doubling	12
4.7	Acceptance criteria	13
5	N02: refinement can coarsen the returned approximation	13
5.1	Both execution paths exhibit the behavior	13
5.2	The implementation is consistent with retargeting, not monotone refinement	14
5.3	Recommended policy and compatibility boundary	14

6	A stronger result for C07: realness is entry-point dependent	15
6.1	Exact is not synonymous with real	15
6.2	A small acceptance matrix isolates the bypass	16
6.3	The source-level explanation	16
6.4	Repair the public contract without breaking legitimate internal complex algebra	17
6.5	Performance and regression details specific to this repair	17
7	A stronger result for C04: native confirmation and a closed-frontier implementation	18
7.1	Current native behavior depends on the requested cutoff	18
7.2	The old diagnosis and the new implementation objective	19
7.3	A precise closed-frontier rule	19
7.4	Sparse implementation and coefficient-generator discipline	20
7.5	Budget semantics and what has actually been tested	20
7.6	Integration recommendation	21
8	Targeted engineering and development recommendations	21
8.1	Prioritize by the claim that can become false	21
8.2	A small implementation sequence with objective exit conditions	22
8.3	Performance measurements that answer these particular questions	22
8.4	Kernel-specific behavior and regression ownership	23
9	Conclusions	23
A	Archive contents and reproduction	24
B	Source-inspection map	25
C	Evidence interpretation	26

1 Results and scope

The repository is not simply a replacement spelling for Series. Its most distinctive component is an explicit computational representation of inverse asymptotics: exact weights, polynomial logarithmic coefficients, selected real approaches, transported uncertainty, retained source equations, and operations on the resulting objects. It also contains specialized coordinate systems and forward adapters that reuse native special-function machinery. The audit therefore treats the mathematical contracts of the objects as seriously as the displayed coefficients.[1, 5, 6, 13]

The principal results are summarized below. The identifiers beginning with Δ refer to the existing consolidated review register; they are not new issue numbers.

ID	Result	Status relative to prior reviews	Evidence added here
N01	Relative-only certificate accuracy stalls at a fixed enclosure precision.	Additional defect; medium priority.	Native failure, two successful controls, and a source-level causal explanation.
N02	A lower-cutoff SeriesRefine call coarsens the returned finite approximation.	Additional API/UX observation; policy decision.	Native forward and inverse examples; both replay and retained-state paths inspected.
Δ C07	Realness validation differs between public entry points.	Existing concern, strengthened.	Four explicit nonreal constants and an acceptance/rejection matrix in a native kernel.
Δ C04	Nonlinear pushforward drops a generated logarithmic boundary degree when the requested cutoff exceeds input precision.	Existing defect, confirmed; high correctness priority.	Native current-snapshot confirmation and an executed sharp-frontier prototype.

1.1 Snapshot and reproducibility boundary

All source claims in this report concern commit 921387e5ba1239bfda96e63e64e89bf63d9c41e6. A successful native load used the public standalone file from that commit. The evaluator reported:

```
15.0.0 for Linux x86 (64-bit) (May 6, 2026)
```

This identifies the execution environment; it is not a statement that 15.0.0 is the newest available Wolfram release. The repository's release and stored validation information must be distinguished from this independently observed environment.[1, 3]

The audit used the connected repository reader for source inspection and a native Wolfram evaluator for focused experiments. Remote service failures occurred on some attempted calls. A service error was not counted as a package failure, a timeout, or an unevaluated mathematical result. Only successful evaluator responses enter the observation tables. The archive's native_observations.json is a

structured manual transcription of those responses, not an automatically captured complete kernel session.

The full repository test suite was not run. The new `DesiredBehavior.wlt` file was authored from the individual witnesses but was not executed as a complete `TestReport`. The Python frontier prototype was executed independently: 13 test methods, including 40 randomized oracle comparisons, passed. The certificate adapter and source patch remain experimental candidates; no successful native execution of their complete artifacts is claimed.

1.2 What the existing reviews remove from this report

The baseline was the repository's consolidated review-status document, the review index, selected original finding sections, and targeted searches of the original review texts. The archive records the paths and successful retrieval sizes. The prior reports mostly analyze an earlier snapshot; the current source and status register already contain focused fixes for the dense native-export allocation problem and the fractional-power-of-pure-uncertainty problem. Those are not presented here as current discoveries.[\[2, 3\]](#)

Likewise, this report does not reissue the existing lists of generic resource-budget inconsistencies, native-export limitations, aggregate test-runner concerns, missing integration or multivariate interfaces, complex/Stokes extensions, generalized transseries plans, and documentation redesigns. Some of those subjects appear briefly in the feature comparison because they define the product boundary; they are not counted as new recommendations.

For N01 and N02, searches across the reviewed article texts for the certificate options and lower-cutoff/coarsening terminology found no corresponding discussion, and neither issue appeared in the consolidated register. Search absence alone is not a proof that no semantically similar sentence exists anywhere in nine long reports. The novelty claim here is narrower: these are additional concrete issues not identified in the inspected register or matched original discussions. By contrast, C04 and C07 are deliberately attributed to the existing review effort. In particular, Review 2 already states the nonlinear-frontier problem, supplies the essential mathematical witness, and proposes a conservative correction.[\[3, 19\]](#)

1.3 Severity is about the contract, not just the output type

A false big-O claim is a correctness failure even when the displayed leading terms are right. Accepting a nonreal expression through a real-only entry point is a domain-contract failure, even if the displayed expression is a perfectly valid complex-valued formula. Returning a conservative failure instead of reaching an attainable accuracy is a different class of defect. Returning a coarser but valid series under an ambiguously named operation is different again.

These distinctions matter for prioritization. C04 should block reliance on affected nonlinear error bounds. C07 needs a consistent realness policy and gate. N01 needs a better accuracy controller, but the failing example does not produce a dishonest certificate. N02 should be resolved as an explicit API policy, without describing its valid output as a mathematical error.

2 The implementation's mathematical and architectural contract

2.1 A positive local coordinate and a sparse coefficient algebra

For a finite source endpoint, the ordinary engine uses a selected positive distance u from that endpoint. At positive or negative infinity it uses a positive reciprocal coordinate. An ordinary precision-tracked jet has the form

$$J(u) = \sum_{\beta \in B} u^\beta P_\beta(\log u) + O(u^P \mathcal{L}(u)^D), \quad \mathcal{L}(u) = 1 + |\log u|, \quad u \rightarrow 0^+, \quad (1)$$

where B is finite, the weights are exact, and each P_β is a polynomial. The internal triple is essentially `{terms, P, D}`. Equal weights must be combined before a boundary coefficient can be declared zero. An exclusive cutoff retains complete blocks below the cutoff, not individual monomials in the logarithm.[5]

The higher-level series calculus uses the more general representation

$$S = \text{Offset} + \text{Prefactor } J(u), \quad (2)$$

with an exact prefactor. The error in (1) is relative to that prefactor. Thus $\exp(x)x^{-1/2}O(x^{-3})$ and $O(x^{-3})$ are not interchangeable. `seriesData`, `seriesFlat`, `seriesMake`, and the arithmetic dispatch maintain this distinction. If no common ordered coefficient algebra can represent compatible operands, the composite path retains an absolute error envelope rather than pretending that a single power cutoff applies.[6, 17, 7]

For two ordinary errors, the coarser error is the one with smaller power P ; at equal power it is the one with larger logarithmic degree D . This is the precision order embodied in `combinePrecision`. The distinction between the power and logarithmic coordinates is central to C04: preserving P while accidentally lowering D can create an unjustified improvement.

2.2 Inverse construction and the role of retained evidence

The ordinary inverse machinery normalizes a forward model to a monomial core with higher-weight corrections. The Lagrange route generates multi-index contributions; the Newton route constructs a unit correction and checks formal residual invariants. The representation retains enough information to inspect blocks, compute selected coefficients, replay an operation, or reuse inverse state. This is more than a finite symbolic formula.[5, 9]

A retained finite forward model is not the same object as the exact original function, and neither is automatically a quantitative error bound at a chosen numerical argument. The package has separate interfaces for a formal inverse residual, a numerical comparison, and an interval certificate. In the inspected numerical checker, `FindRoot` produces a reference root and the result explicitly identifies itself as numerical evidence. The certificate engine instead uses exact rational interval operations and elementary-function tail enclosures over a supplied interval.[5, 10, 8]

A potentially serious objection was examined and rejected during this audit: a magnitude-only declared input remainder would not by itself establish eventual invertibility. However, the user guide's `"InputRemainder" -> {rho, k}` declaration includes both the value order and the matching first-derivative order in the positive source coordinate. Treating that option as if it declared only a value bound would mischaracterize the current documented contract.[4]

2.3 The specialized engines are not one universal series ring

The inspected modules implement several distinct representations. Gamma and Barnes forward products are normalized logarithmically, with positivity checks and an extracted carrier. Their inverse engines use an exact Lambert-based core and inverse-core blocks with reciprocal-logarithmic coefficients. Those blocks are not ordinary Taylor terms in the original target.[11, 12, 1]

Finite flat sectors use a separate exponential-sector degree and inner power cutoff. Their operations transport inner errors and the omitted-sector tail separately. Fourier-log inverse coefficients are finite sums of polynomial amplitudes times exact logarithmic-frequency modes. Their internal amplitudes may be complex, but conjugate symmetry is checked to establish the realness of the whole coefficient. These facts constrain the realness repair discussed later: a blanket ban on every internal complex coefficient would break a legitimate representation.[15, 16]

The native special-function adapter is also significant. It obtains structured native expansions, traverses their `SeriesData` trees, separates carriers and bounded oscillatory modes, and tracks absolute errors. When a native result has no `SeriesData`, the inspected path requires exact equality with the source before accepting it as an exact finite result. Real projection is preceded by a source-realness check. The adapter therefore should not be described as inventing the Bessel, Airy, or related asymptotics independently of Wolfram’s built-ins. Its additional work lies in interpretation, domain checks, representation, and subsequent calculus.[13, 14]

2.4 Boundaries checked without inventing additional bugs

Several inspected guardrails are worth retaining. Differentiation of a nonzero big- O remainder requires an appropriate derivative contract; a magnitude statement alone is not silently differentiated. The certificate domain checker verifies retained source conditions on the whole closed interval, not merely at the seed. The native importer distinguishes unsupported operations from exact finite results. Exact source-coordinate reconstruction uses explicit exactness evidence or a symbolic composition check before retaining an exact inverse carrier.[6, 8, 13, 18]

These observations are not a proof of the entire package. They explain why this report does not promote every suspicious-looking fallback or metadata field into a new finding. They also identify behavior that a targeted fix should preserve.

3 Comparison with Wolfram Language built-ins

3.1 The documented boundary

Native `Series` handles much more than integer Taylor polynomials: its documented scope includes negative and fractional powers and logarithmic behavior. `SeriesData` uses a rational exponent lattice, permits slowly varying functions in coefficients, and can appear inside expressions with external symbolic powers, rapid carriers, or oscillatory factors. Consequently, “not a single ordinary `SeriesData`” does not imply “not expressible by native series machinery.”[20, 21]

`InverseSeries` and `ComposeSeries` are the direct native tools for formal reversion and composition of suitable series. `AsymptoticSolve` operates at the equation level: its documented scope includes

systems and real or complex solutions with automatically inferred scales. `Asymptotic` likewise is not restricted to ordinary Taylor scales. This package instead concentrates on selected scalar real inverse families and on preserving explicit generalized-series data.[22, 23, 24, 25]

Capability	Native point of comparison	Package distinction at the audited snapshot
Ordinary local expansion	<code>Series</code> , <code>SeriesCoefficient</code> .	Explicit complete-block cutoff and a retained power/log error pair. Considerable mathematical overlap.
Formal inverse	<code>InverseSeries</code> ; <code>AsymptoticSolve</code> . local	Exact generalized weights, logarithmic-polynomial blocks, a source-side choice, and inverse provenance.
Implicit systems	<code>AsymptoticSolve</code> .	Not a general systems solver; its scalar inverse specialization is a deliberate boundary.
Composition	<code>ComposeSeries</code> ; native series substitution.	Explicit transported uncertainty in admitted generalized coordinates; other compatible arithmetic can use composite envelopes.
Elementary logarithmic/exponential inversion	Native algebra and <code>ProductLog</code> , with equation-level asymptotic tools.	Specialized coordinate and exact-core reconstruction plus retained correction data.
Gamma/Barnes forward	Native special-function asymptotics.	Logarithmic product normalization and exact prefactors, with cutoff semantics in a correction bracket.
Gamma/Barnes inverse	Equation-level asymptotics and exact Lambert cores are relevant native ingredients.	Dedicated inverse-core scales and family-specific checks. No comprehensive native inverse comparison was executed here.
General special-function forward expansion	Native <code>Series/Asymptotic</code> .	Structured importer and real-domain/error-envelope interpretation; not an independent catalogue of all formulas.
Flat and Fourier-log corrections	Native exponential/trigonometric expressions and asymptotic machinery.	Separate finite-sector or exact-frequency coefficient data, with explicit budgets and remainder conventions.
Certified inverse value	Numerical root finding is not by itself a proof.	A narrow explicit rational enclosure algorithm for admitted elementary expressions, separate from formal asymptotic accuracy.

The package column follows the cited source and guide; the native column describes interfaces, not universal success on a family. Integration and differential-equation extensions already belong to the existing backlog and are not new recommendations here.[4, 5, 8, 3]

3.2 Executed comparison B01: ordinary polynomial reversion

The native calls

```
InverseSeries[Series[x + x^2, {x, 0, 4}], y]
AsymptoticSolve[x + x^2 == y, {x, 0}, {y, 0, 4}, Reals]
```

returned, respectively, a native series and a solution rule with finite part

$$y - y^2 + 2y^3 - 5y^4. \quad (3)$$

The InverseSeries result was

```
SeriesData[y, 0, {1, -1, 2, -5}, 1, 5, 1]
```

The package call with exclusive cutoff 5 produced the same finite part and remainder power 5. This is a clean overlap case. There is no reason to attribute the coefficients themselves to a capability missing from the native system. A caller may nevertheless prefer the package's retained source, branch, and refinement data.

The translation from package cutoff 5 to native order 4 in this example is specific to its integer grid. It should not be applied mechanically to irrational weights, factored corrections, or marker-depth APIs.

3.3 Executed comparison B02: a logarithmic inverse

For the positive germ of $f(x) = x + x^2 \log x$ near zero, the package call

```
AsymptoticInverse[x + x^2 Log[x], {x, 0}, {y, 4}]
```

returned the finite part

$$y - y^2 \log y + y^3 (\log y + 2 \log^2 y). \quad (4)$$

The tested native call

```
AsymptoticSolve[x + x^2 Log[x] == y,
  {x, 0}, {y, 0, 3}, Reals]
```

returned unevaluated in the successful evaluation response. It was not reported as a timeout.

This observation supports a practical statement about the tested interface and kernel: the package directly supplies the requested generalized inverse blocks where that particular native equation-level call did not. It does not establish that no combination of native transformations, formal coefficient calculations, alternative options, or future kernels can obtain (4). The package itself makes extensive use of native symbolic operations.

3.4 Executed comparison B03: an irrational correction weight

For $f(x) = x + x^{\sqrt{2}}$ on the positive side, the package's exclusive cutoff 2 returned

$$y - y^{\sqrt{2}} + \sqrt{2} y^{2\sqrt{2}-1}. \quad (5)$$

The tested native call

```
AsymptoticSolve[x + x^Sqrt[2] == y,
  {x, 0}, {y, 0, 3}, Reals]
```

again returned unevaluated. The observation is bounded by that exact invocation.

The package output is independently consistent with first-order reversion algebra. Set $q = \sqrt{2}$ and $x = y + Ay^q + By^{2q-1} + \dots$. Expanding x^q about y gives $A = -1$ and $B = q$. The exponents $1, q, 2q - 1$ are all below 2, while the next simple iterated weight $3q - 2$ exceeds 2. This checks the displayed terms without relying on the package's own inverse residual routine.

3.5 Executed comparison B04: the Stirling carrier

The package call

```
AsymptoticExpansion[Gamma[x], {x, Infinity, 3}]
```

returned

$$\frac{\sqrt{2\pi}}{\sqrt{x}} e^{x(\log x - 1)} \left(1 + \frac{1}{12x} + \frac{1}{288x^2} \right). \quad (6)$$

The native call

```
Asymptotic[Gamma[x], x -> Infinity, SeriesTermGoal -> 3]
```

returned the same three terms distributed across separate summands. For $x > 0$, their equivalence follows immediately from $e^{x \log x} = x^x$. Here the package's useful distinction is the explicit retained carrier and its correction-bracket contract, not a previously unavailable asymptotic formula.

3.6 What these experiments do not establish

No speed ranking follows from these calls. Network latency, remote service behavior, source download, kernel setup, and symbolic caches were not controlled as a benchmark environment. The archive's reproduction runner records elapsed time for future local investigation, but this report makes no runtime claim from those unexecuted aggregate records.

Nor should a successful native finite expression be silently assigned the package's richer error metadata. Equality of finite parts, equality of branch conditions, equality of retained scales, and equality of error claims are separate comparisons. The four cases above explicitly distinguish a native series object, native solution rules, an unevaluated call, and an expanded carrier expression. That distinction is more informative than a binary "supported/unsupported" checklist.

4 N01: certificate accuracy can stagnate at fixed arithmetic precision

Additional finding. Reproducible accuracy-adaptation failure, not an unsound certificate.

Location. `InverseCertificates.wl`: automatic order selection in `InverseCertificate`, followed by its successful-but-insufficient refinement branch.

Priority. Medium for correctness-preserving usability; potentially high for workflows that depend on requested relative accuracy.

4.1 A small exact problem separates the failure modes

Use the inverse of x^2 on the positive infinite source branch. The resulting inverse expression is simply $\sqrt{y}$, so this experiment does not need a difficult asymptotic reversion:

```
s = AsymptoticInverse[x^2, {x, Infinity}, {y, 1}];
r = InverseCertificate[s, 2,
  "Interval" -> {1, 2},
  "RelativeError" -> 10^-120,
  "MaxRefinements" -> 3,
  "RefineExpansion" -> False];
```

The successful native evaluation returned `Failure["AccuracyNotReached", ...]`. Its history contained four entries—the initial attempt and three refinements—with enclosure orders

$$60, 60, 60, 60. \quad (7)$$

The best certificate's absolute error bound was approximately

$$7.2797457561222431 \times 10^{-85}. \quad (8)$$

The object still contained a valid interval certificate; the requested relative accuracy had not been reached. This is exactly the distinction conveyed by the failure tag.[\[26\]](#)

Two controls are decisive. Keeping the relative tolerance but setting `"EnclosureOrder" -> 150` succeeded. Replacing the relative tolerance with `"TargetError" -> 10^-120`, while leaving the enclosure order automatic, also succeeded.

Request	Order history	Accuracy reached	Best/final absolute error bound
Relative 10^{-120} , automatic order	60, 60, 60, 60	No	7.27975×10^{-85}
Relative 10^{-120} , explicit order 150	150, 150, 150, 150	Yes	3.09969×10^{-193}
Absolute 10^{-120} , automatic order	135, 135, 135, 135	Yes	3.57370×10^{-175}

All three use the same function, target, verification interval, refinement budget, and disabled expansion refinement. At a root of magnitude $\sqrt{2}$, absolute and relative goals of this size differ only by a factor of

order one. A roughly 10^{90} discrepancy in the resulting error floor is not explained by that distinction. It is explained by the different arithmetic precision selected internally.[26]

The absolute request is a control, not a claim that the public options have identical semantics. The relative-only request needs an error of order $\sqrt{2} 10^{-120}$; the absolute control asks for 10^{-120} and is slightly more demanding here.

4.2 The first cause: relative tolerance does not size the arithmetic

When "EnclosureOrder" is automatic, the constructor computes a digit estimate from "TargetError" only. If no absolute tolerance was supplied, that estimate is zero. It then chooses

$$n = \max(\text{WorkingPrecision} + 10, \text{digits} + 15). \quad (9)$$

With the default working precision 50, the relative-only experiment starts at $n = 60$. With absolute tolerance 10^{-120} , the source's integer-length calculation selects $n = 135$. The exact observed order histories agree with this source path.[8]

The interval context uses $4(n + 10)$ significant binary bits for directed dyadic rounding. At $n = 60$, this is 280 bits. A unit-scale rounding quantum of order 2^{-280} is of order 10^{-84} , consistent with the observed stagnation near (8). This is an explanatory scale estimate, not a universal theorem that every computation at 280 bits must have exactly that error floor. Exact cancellations, exponent sizes, and enclosure dependency affect the actual result.

4.3 The second cause: a successful proof is mistaken for sufficient arithmetic progress

The adaptive loop has two different paths after a certificate attempt. When an association is returned and the center is not fixed, it records that a root is known, shrinks the verification interval to the root enclosure when possible, and updates the center. It can also seek a better asymptotic seed. However, this path does not increase the enclosure order. The order is doubled on the other path.[8]

This asymmetry matters because a successful certificate can be too wide. Once interval arithmetic is the limiting factor, shrinking a verification interval at the same rounded precision may no longer improve the proof to the requested accuracy. The history continues to report successful certification attempts, while the accuracy test continues to fail. A smaller geometric interval is not a substitute for more accurate endpoint arithmetic.

There are therefore two cooperating defects in the adaptation policy: the relative tolerance is absent from initial planning, and the successful-but-insufficient path cannot escape the arithmetic precision it chose. Either can remain hidden in ordinary tests. The successful absolute control exercises a different initial-order path and does not test the relative-only behavior.

4.4 Why this does not undermine the interval proof itself

For an admitted explicit function, the certificate attempt encloses the derivative and obtains a positive lower bound μ on its absolute value. It encloses the residual at an exact rational center c , obtaining a bound ε . A mean-value estimate then gives a candidate root radius

$$r = \varepsilon / \mu. \quad (10)$$

The implementation checks the branch and domain on the interval and uses containment or exact endpoint sign information to establish existence; a signed derivative enclosure provides a sharper root enclosure. Subsequent attempts can reuse already established root existence.[8]

N01 does not identify a wrong derivative interval, a wrong residual interval, or a false existence claim. It identifies a controller that keeps proving something weaker than the user requested. The current failure result is conservative. A repair should preserve that conservatism: no heuristic precision estimate, numerical seed, or favorable decimal comparison should be allowed to manufacture "AccuracyGoalReached" -> True.

4.5 A targeted repair and the supplied candidates

A minimal source change has two parts. First, let the automatic initial order depend on a digit estimate for both positive exact tolerances. Second, permit enclosure-order growth when a successful certificate still misses the accuracy goal. The supplied `patch_certificate.py` generates precisely this experimental change in copies of both the kernel module and the standalone file.

The patcher is intentionally restrictive. It checks the pinned Git blob identities, allowing CRLF-to-LF normalization, requires exactly one occurrence of each source anchor, refuses a nonempty destination, and writes outside the checkout. It does not edit the user's repository or update Git state. In an integration checkout, the standalone should ultimately be rebuilt through the repository's normal generator rather than maintained as an independent hand-edited implementation.

The archive also supplies `AccuracyAwareCertificate.wl`, an opt-in adapter that leaves the original definitions unchanged. It chooses a tolerance-aware starting order. Only failures tagged `AccuracyNotReached` are retried with a higher initial order. The original `InverseCertificate` remains the sole producer of any mathematical certificate. Domain failures, unsupported expressions, and fixed-center accuracy-floor failures are not converted into success or indiscriminately retried.

These two artifacts have different roles. The source patch repairs the internal successful-attempt path and can reuse its interval progress. The adapter is an isolated workaround candidate; restarting an entire certificate call can redo work. Neither complete artifact was validated natively in this audit. The original package's explicit-order workaround, in contrast, was actually executed and succeeded.

4.6 A better precision controller than unconditional doubling

Unconditional doubling is a useful minimal candidate, not the final performance design. A more discriminating controller should distinguish three quantities:

$$\text{seed error,} \quad \text{geometric verification width,} \quad \text{arithmetic enclosure width.} \quad (11)$$

This separation can remain private to the certificate implementation; it does not require a new general series architecture. After a successful but inaccurate attempt, record the residual-enclosure width and the root-enclosure width. If geometric contraction stalls while the residual width changes little, raise the arithmetic order. If the residual enclosure is already narrow and the center remains far from the root, improve the center or use the interval midpoint. A proved fixed-center accuracy floor should still terminate immediately.

The initial decimal digit estimate is only a starting heuristic. For a very large or very small root, a relative tolerance maps to a different absolute target. Once a certified positive lower bound on $|x_*|$ is available, use the existing sufficient absolute tolerance derived from it. The result must continue to be accepted only by the exact certificate inequality, not by the planning estimate.

The current source caps the enclosure order at 2000. The proposed controller should report which limit was active when stopping: maximum order, maximum refinements, unsupported enclosure, fixed-center error floor, or no certified interval. This is a targeted explanation of N01's failure state, not a restatement of the prior repository-wide resource-budget redesign.

4.7 Acceptance criteria

The original relative-only witness must reach its goal under an appropriately documented adaptive budget, and the absolute and explicit-order controls must remain successful. Additional tests should include a fixed center with a genuine accuracy floor, a root close to zero with an explicit absolute fallback, a relative tolerance at several root magnitudes, and a request that intentionally exhausts the enclosure-order budget. The success predicate must remain an exact statement about the returned certificate.

For performance, count certificate attempts, enclosure orders, and reused verification intervals before measuring wall time. An implementation that fixes N01 by restarting many expensive enclosure calculations may be correct but unnecessarily costly. The archive records the observed order histories because they identify the controller defect without requiring a noisy remote timing benchmark.

5 N02: refinement can coarsen the returned approximation

Additional observation. A concrete naming and policy ambiguity; no invalid asymptotic formula is alleged.

Locations. `SeriesOperations.wl`, the forward-source replay path of `SeriesRefine`; `RefinementState.wl`, `refineStoredInverse`.

Decision needed. Does “refine to h ” mean “at least h ” or “retarget to exactly the truncation policy h ”?

5.1 Both execution paths exhibit the behavior

The forward example is direct:

```
s = AsymptoticExpansion[Sin[x], {x, 0, 7}];
t = SeriesRefine[s, 3];
{Normal[s], s["RemainderPower"], Normal[t], t["RemainderPower"]}
(* {x - x^3/6 + x^5/120, 7, x, 3} *)
```

The inverse example reaches the retained-state implementation rather than merely replaying a forward source:

```
s = AsymptoticInverse[x + x^2, {x, 0}, {y, 5}];
t = SeriesRefine[s, 3];
{Normal[s], s["RemainderPower"], Normal[t], t["RemainderPower"],
 t["RefinementStatistics"]["Strategy"]}
(* {y - y^2 + 2 y^3 - 5 y^4, 5,
    y - y^2, 3, "IncrementalLagrange"} *)
```

Both outputs were observed in successful native evaluations.[\[26\]](#)

The original source is still retained, and an inverse computation state may retain additional blocks. Therefore this is not a claim that all information in the object has been irreversibly destroyed. It is a claim that the returned finite approximation and its advertised remainder become coarser under an operation named `SeriesRefine`.

5.2 The implementation is consistent with retargeting, not monotone refinement

The forward path calls the constructor again using the requested cutoff. The ordinary inverse path has an unchanged-cutoff fast path, but no general lower-cutoff refusal or no-op. It trims available state blocks at the requested cutoff and assembles the result accordingly. That explains why both examples coarsen and why the inverse result can truthfully report incremental reuse while displaying fewer terms.[\[6, 9\]](#)

This is algebraically legitimate. The sine approximation $x + O(x^3)$ remains true, as does the coarser inverse approximation. The issue is the observable contract for callers. A client accumulating precision requests may reasonably regard refinement as monotone, especially because the package also exposes `SeriesTruncate`. A stale lower request can then unexpectedly replace a better displayed approximation with a worse one.

For a forward source, the current behavior can additionally do work to produce a coarser answer even when the original object already meets the requested lower accuracy. No timing claim is necessary to identify that avoidable replay path. The existing retained-state path is better positioned to avoid recomputation, but it still chooses a coarser presentation.

5.3 Recommended policy and compatibility boundary

The recommended policy is: *within a comparable representation, a refinement request is a minimum information request*. A lower or equal requested cutoff should return the current approximation, optionally with no-op statistics. An explicit `SeriesTruncate` call should be the operation that discards displayed blocks. An alternative, equally coherent policy is to reject lower cutoffs with a dedicated failure. The archive's desired-behavior test permits either no coarsening or an explicit failure.

This change should not be implemented by taking a numeric maximum of arbitrary "Cutoff" fields across different scale families. A logarithmic depth, an inverse-core exponent, and an exponential-sector depth need not be comparable. The simple guard applies only when the constructor or retained representation establishes the same coordinate and cutoff convention. Other cases need their own local policy, or a documented retargeting operation.

A compact implementation outline for the ordinary comparable path is:

```
validateRequestAndRepresentation[s, h];
If[sameCutoffConventionQ[s, h] && requestAlreadySatisfiedQ[s, h],
  Return[withNoOpRefinementStatistics[s, h]];
(* Continue through the existing state/replay implementation. *)
```

This is a design sketch, not a drop-in replacement. The implementation must distinguish an already satisfied request from an invalid one, preserve the retained domain, and not falsely assert comparability. It should also preserve exact finite termination: an exact result needs no additional coefficient work merely to satisfy a nominal larger cutoff.

The compatibility cost is explicit. A caller using `SeriesRefine[s, smallerH]` as an implicit truncation operation would need to use `SeriesTruncate`. Before changing behavior, the maintainers should decide whether such usage is intentionally supported. Documentation alone can resolve the ambiguity by declaring retargeting semantics, but that would retain the stale-request hazard and the distinction from ordinary meanings of refinement.

6 A stronger result for C07: realness is entry-point dependent

Existing concern, materially stronger evidence. C07 in the consolidated register already identifies inconsistent real-coefficient validation. This section adds concrete native witnesses and a cross-entry-point test matrix. It does not claim discovery of the general concern.

Contract at risk. A real-asymptotic interface admits exact but nonreal constants on some paths while rejecting them on another.

6.1 Exact is not synonymous with real

The current exactness predicate is syntactic: it excludes inexact real atoms and explicit complex-number atoms. A symbolic exact expression can nevertheless have a nonreal value. For example, with a real $a > 0$,

$$\sqrt{-a} = i\sqrt{a}, \quad \log(-a) = \log a + i\pi \quad (12)$$

on principal branches. Likewise, $\arcsin 2$ and $\arccos 2$ are nonreal. None of these statements depends on numerical roundoff.

The following forward calls all returned `GeneralizedSeries` in the native evaluator:

```
AsymptoticExpansion[Sqrt[-a] + x, {x, 0, 2},
  Assumptions -> a > 0]
AsymptoticExpansion[Log[-a] + x, {x, 0, 2},
  Assumptions -> a > 0]
AsymptoticExpansion[ArcSin[2] + x, {x, 0, 2}]
AsymptoticExpansion[ArcCos[2] + x, {x, 0, 2}]
```

Their finite parts were the expressions supplied, and the reported "TargetDomain" in each observation was $x > 0$. The parameter assumptions are separate metadata; the displayed approach condition does not turn a complex constant into a real one.[\[26\]](#)

The returned expressions are not wrong as complex formulas. The defect is accepting them through a workflow whose other operations enforce a real-coefficient contract, without making a corresponding change of result kind or domain policy.

6.2 A small acceptance matrix isolates the bypass

Let

```
s = AsymptoticExpansion[x, {x, 0, 3}];
```

The following outcomes were observed:

Public call	Native outcome
SeriesAdd[s, ArcSin[2]]	Failure with tag UnprovedRealCoefficient
SeriesObservable[s, z + ArcSin[2], z]	Accepted; finite part $x + \arcsin 2$.
SeriesObservable[s, ArcSin[z + 2], z]	Failure with tag UnsupportedObservable

This is stronger than merely observing that an exactness predicate is syntactic. The same constant is rejected as a regular arithmetic operand but accepted when embedded in an observable. It supplies a concrete regression target for the actual public dispatch paths.[\[26\]](#)

6.3 The source-level explanation

In the forward dispatcher, a subexpression independent of the local variable is sent to pConst. That helper checks polynomial dependence on the logarithmic coefficient variable and whether the coefficient is zero; it does not establish realness of an arbitrary exact constant. The ordinary observable walker has a corresponding independent-expression route.[\[5, 6\]](#)

By contrast, seriesRegularOperand checks that the resulting coefficient polynomials are provably real under the relevant assumptions. This is the gate reached by SeriesAdd[s, ArcSin[2]]. The mismatch is thus not explained by different mathematical branches of the same function. It is explained by different validation paths for constants that enter the same intended real series calculus.[\[7\]](#)

The parameterized special-function path provides an instructive existing pattern: it checks source realness independently and verifies the realness of the resulting coefficient polynomials. The native importer similarly proves realness before taking a real projection. These are useful local precedents for a repair, not evidence that every forward path already has the same protection.[\[14, 13\]](#)

6.4 Repair the public contract without breaking legitimate internal complex algebra

A repair should separate three questions: is the original function real on the selected approach; are the retained coefficients in the admitted coefficient algebra; and has the chosen normalization preserved that function and domain? A successful native Taylor calculation answers none of those questions automatically for every possible source expression.

For ordinary real power-log results, validate the canonicalized coefficient blocks under explicit assumptions and a real logarithmic coordinate. For variable-independent regular operands, apply the same realness gate consistently whether they arrive through arithmetic or an observable. For a forward source with special-function or branch-sensitive structure, retain the source-domain check rather than treating a real finite truncation as a proof that the original source is real.

The distinction matters even beyond the concrete witnesses. As a general mathematical example, a real formal power series can be the power-series part of a function with an exponentially small imaginary component. Real retained coefficients alone do not prove realness of the source germ. This is a conceptual requirement for the validator, not an additional observed package failure.

There should be three outcomes for a proof attempt: proved real, proved nonreal, and unresolved under the supplied assumptions. A known nonreal source should receive a direct domain failure. An unresolved coefficient should receive a failure or a documented conditional result, not silent admission because it contains no explicit complex-number atom. The existing real arithmetic path already makes a conservative choice for an unproved coefficient; the objective is consistency across entry points.

The validator should operate after appropriate exact simplification and complete-block merging. Rejecting every syntactically nonreal subexpression before allowing exact cancellation can be overly restrictive. For example, real combinations can be represented using complex intermediate terms. More importantly, the Fourier coefficient engine deliberately uses complex modes with a checked conjugacy relation. A patch to pConst that indiscriminately bans all internal complex coefficients would confuse the ordinary real coefficient algebra with that distinct Fourier algebra.^[16]

The appropriate implementation boundary is therefore not “all constants everywhere must be individually real.” It is “each public real result must carry a valid real-source/real-representation justification appropriate to its algebra.” The immediate ordinary-path fix can be small, but it should explicitly exclude the internal Fourier representation from a scalar-only test.

6.5 Performance and regression details specific to this repair

Repeated symbolic realness checks can be expensive. Within one operation, canonicalize equal constant expressions and reuse a result keyed by the coefficient expression and the assumptions/domain used in that check. A global cache that ignores the proof context would be unsafe. This is a targeted optimization for the additional gate, not a general recommendation to memoize every simplification in the package.

The minimum regression matrix should cover the four witnessed nonreal constants, the same constants in direct scalar arithmetic and observables, a positive real algebraic constant, a parameter declared real, a parameter whose realness is unresolved, and a Fourier coefficient represented by conjugate complex modes. Tests should distinguish a conservative unresolved failure from acceptance of a

known nonreal value. A broad real-projection fix that makes the complex part disappear would be incorrect even if it made the test output look real.

No source patch for this policy is supplied as production-ready code. The desired-behavior tests record the unambiguous known-nonreal cases. The treatment of general unresolved coefficients and valid complex cancellations needs an explicit maintainer decision before a broader patch is integrated.

7 A stronger result for C04: native confirmation and a closed-frontier implementation

Previously diagnosed defect. Review 2 already identifies this nonlinear error-bound problem and supplies its essential proof and conservative correction. The additions here are current-snapshot native confirmation and a tested implementation of complete-boundary accounting. The boundary idea itself was already suggested as an optimization in that review.[\[19\]](#)

7.1 Current native behavior depends on the requested cutoff

Construct an input whose retained part is $x \log x$ and whose omitted explicit term has order x^2 :

```
s = AsymptoticExpansion[x Log[x] + x^2, {x, 0, 2}];
```

Its stored information is $x \log x + O(x^2)$. The default exponential operation returned

$$1 + x \log x + O(x^2 \mathcal{L}(x)^2). \quad (13)$$

Requesting a larger cutoff changed the error degree:

```
t = SeriesExp[s, 3];
{Normal[t], t["Remainder"]}
(* {1 + x Log[x], PowerLogRemainder[x, 2, 0]} *)
```

The same larger-cutoff behavior was observed for the logarithm of `SeriesAdd[s, 1]` and for its square root.[\[26\]](#)

Operation at cutoff 3	Observed finite part	Observed error descriptor
<code>SeriesExp[s, 3]</code>	$1 + x \log x$	$O(x^2)$
<code>SeriesLog[SeriesAdd[s, 1], 3]</code>	$x \log x$	$O(x^2)$
<code>SeriesPower[SeriesAdd[s, 1], 1/2, 3]</code>	$1 + \frac{1}{2}x \log x$	$O(x^2)$

The essential contradiction is already in Review 2. For the exponential witness,

$$e^{x \log x + x^2} - (1 + x \log x) = x^2 \left(1 + \frac{1}{2} \log^2 x\right) + o(x^2), \quad (14)$$

so division by x^2 is unbounded as $x \rightarrow 0^+$. A native `Series` control independently returned the displayed second-order coefficient $(2 + \log^2 x)/2$. The point of repeating this small formula is to identify exactly what the native package confirmation tests; it is not a new proof of the known defect.[19, 26]

7.2 The old diagnosis and the new implementation objective

The source helper `unitSeriesPrecision` sets the active boundary $c = \min(P_U, h)$. When $P_U < h$, it can return the input error degree D_U alone. The finite Taylor calculation discards powers at the exclusive boundary, so a known nonlinear term landing exactly at P_U can disappear without its logarithmic degree entering the error.[5]

Review 2's conservative correction includes both the transported input degree and a generated Taylor degree bound. That is an appropriate immediate correctness repair. The implementation supplied here explores the sharper version: compute the complete polynomial at the active boundary, use its actual degree, and only then remove it from the finite output.

This avoids a particular overestimate. For

$$U(u) = uL + u^{3/2}L^{10}, \quad L = \log u, \quad R = O(u^2), \quad (15)$$

the boundary at u^2 in e^U is $\frac{1}{2}L^2$. A global estimate based on two Taylor powers times the maximum input logarithmic degree gives 20, whereas the complete boundary degree is 2. The L^{10} term occurs at a higher weight and cannot contribute to the weight-2 boundary through a quadratic product. This is a useful accuracy improvement without claiming additional known powers of u .

7.3 A precise closed-frontier rule

Let

$$U(u) = \sum_{\alpha \in A} u^\alpha P_\alpha(L), \quad a = \min A > 0, \quad (16)$$

and let the unknown input error satisfy

$$R(u) = O(u^P \mathcal{L}(u)^D), \quad P > \max A. \quad (17)$$

The exact-input case is represented by $P = +\infty$. Let Φ be analytic near zero with fixed Taylor coefficients c_k , and let $h > 0$ be the requested exclusive cutoff. Set $c = \min(P, h)$, assumed finite in the prototype, and form

$$T_{\leq c}(u) = \left[\sum_{k=1}^{\lfloor c/a \rfloor} c_k U(u)^k \right]_{\leq c}. \quad (18)$$

Write $\Psi_c(L)$ for the complete coefficient of u^c in this expression, or zero if no such weight occurs. Return the terms strictly below c , but assign logarithmic degree

$$D_{\text{out}} = \max(\deg_+ \Psi_c, D \mathbf{1}_{P=c}), \quad (19)$$

where $\deg_+ 0 = 0$. The constant $\Phi(0)$ is handled separately.

Proposition 1. *Under the stated assumptions, the returned finite part $T_{<c}$ satisfies*

$$\Phi(U + R) = \Phi(0) + T_{<c} + \mathcal{O}(u^c \mathcal{L}(u)^{D_{\text{out}}}). \quad (20)$$

Proof. For fixed parameters, $U \rightarrow 0$ and $R \rightarrow 0$. Analyticity gives $\Phi(U + R) - \Phi(U) = \mathcal{O}(R)$ in a sufficiently small neighborhood. If $P = c$, this contributes degree D at the active boundary. If $P > c$, its additional positive power absorbs every fixed logarithmic factor and it is $\mathcal{O}(u^c)$.

The Taylor remainder after degree $m = \lfloor c/a \rfloor$ is $\mathcal{O}(U^{m+1})$. Since $(m + 1)a > c$, it too is $\mathcal{O}(u^c)$ after absorbing fixed logarithmic powers. Among the finitely many terms in the retained Taylor polynomial, every weight strictly greater than c has the same property. The only discarded known term that can require a nonzero logarithmic degree at weight c is $u^c \Psi_c(L)$. Combining its degree with the active input error gives (19). $\square$

The rule deliberately does not infer exact termination from a zero boundary polynomial. The true next term can occur at a later weight, and an active input error remains unknown even if all known boundary contributions cancel. The prototype therefore returns a conservative power- c error unless a separate stronger contract is provided. Nor does the proposition assert a uniform parameter-dependent constant: analyticity and its neighborhood are pointwise assumptions here.

7.4 Sparse implementation and coefficient-generator discipline

The file `closed_frontier.py` implements (18) using exact rational weights and symbolic polynomial coefficients. It multiplies sparse powers while retaining weights *at or below* the boundary. Products exceeding the boundary are pruned before coefficient multiplication. Equal weights are merged before the boundary polynomial is inspected. The returned finite part is then selected with the original strict inequality.

The distinction between closed and open truncation is an internal calculation convention, not a public cutoff change. The public result still excludes the boundary. The extra coefficient exists only to justify the error assigned to the terms that were removed.

There is an important implementation detail when porting this approach into the Wolfram code. Some coefficient generators maintain recurrence state. A repair must not calculate the finite series using one sequence of calls and then call the same stateful generator again from $k = 1$ to obtain a boundary coefficient. That can advance the recurrence twice or reuse stale state. The prototype requests each c_k once, in increasing order, including a possible final power whose only retained term lies on the boundary. A `None` return denotes permanent termination, analogous to the existing generator convention's termination marker.

For example, when $U = uL$ and $c = 4$, the prototype calls the generator with 1, 2, 3, 4 exactly once each. The original exclusive finite part needs only the first three powers, but the fourth must be available for boundary accounting. The corresponding test verifies the call sequence as well as the coefficient result.

7.5 Budget semantics and what has actually been tested

The prototype uses a single counter for admitted sparse pair products across the pushforward operation. Exceeding that counter raises a controlled `BudgetExceeded` exception. It rejects floating-

point weights, nonpositive input weights, and known input terms at or beyond a finite declared error frontier. Coefficients must be polynomials in the designated logarithm symbol, and Taylor coefficients must not depend on that symbol.

The code is intentionally not a reimplement of the whole package. It supports rational weights, not the repository's complete exact-real comparison machinery; it does not infer analyticity, source realness, parameter uniformity, or derivative contracts. Its role is to make the proposed boundary calculation executable and independently testable.

The executed tests include the exponential, logarithm, and square-root witnesses; the sharp degree-2 example (15); a nonresonant frontier; preservation of an unknown input error after known coefficient cancellation; ordered single-use coefficient calls; permanent termination; budget failure; and invalid-input cases. A further test uses 40 deterministic randomized cases with seed 20260909. It substitutes $u = t^2$ and compares the sparse result with a separately constructed ordinary polynomial in t . The oracle checks both the strict finite part and the complete boundary polynomial. All 13 test methods passed.[27]

This is independent validation of the rational-weight prototype, not a passing Wolfram package regression suite and not a formal proof of an integrated source patch. The proposition above supplies the mathematical justification; the tests exercise the concrete implementation within its stated scope.

7.6 Integration recommendation

The conservative degree correction already proposed in Review 2 is the lower-risk immediate repair. The closed-frontier calculation can then be introduced as an optional sharper path with a hard budget. If that sharper path cannot complete, it must fall back to the proved conservative degree bound or return a controlled resource failure, never to the known unsound input-degree-only branch.

The integration tests should compare both paths on the same inputs. The sharp result may have a lower logarithmic error degree, but its retained finite part below the common boundary must agree, and its bound must remain valid when unknown input errors are present. A zero known boundary polynomial must not erase an independent input remainder. These are concrete acceptance conditions for the supplied implementation delta, not a reissued general request for more tests.

8 Targeted engineering and development recommendations

8.1 Prioritize by the claim that can become false

The existing C04 error-bound defect and the now-explicit C07 realness inconsistency should be addressed before adding more nonlinear operations that reuse the same low-level paths. This does not make them new backlog entries. The contribution here is that they now have native current-snapshot witnesses and a specific cross-entry or frontier test corpus.

N01 should be addressed with a separate certificate-controller change. It is valuable to keep that change isolated from the interval arithmetic itself: the same exact proof engine can continue to establish every successful bound, while the planner becomes better at choosing enough precision. N02 should be resolved as an API decision rather than smuggled into an arithmetic bug fix. The

choice between minimum-precision refinement and retargeting has compatibility implications that deserve an explicit release note.

8.2 A small implementation sequence with objective exit conditions

Step	Change	Exit condition
1	Integrate a sound nonlinear frontier degree rule; retain the existing conservative option.	The three native C04 witnesses never report degree 0 at power 2; the frontier is computed or conservatively bounded before it is discarded.
2	Make ordinary realness validation consistent across forward constants, scalar operands, and observables.	The witnessed nonreal constants are rejected consistently, while proved-real inputs and conjugate-symmetric Fourier data retain their intended behavior.
3	Add relative-aware certificate precision planning and escalation after insufficient successful proofs.	N01 succeeds under the chosen documented budget; domain and fixed-center failures remain conservative; order histories explain progress.
4	Decide and implement lower-cutoff refinement semantics.	The forward and inverse N02 examples either preserve the current approximation, reject the lower request, or are explicitly documented as intentional retargeting.
5	Port the sharp closed-frontier calculation behind a bounded fallback.	It matches the conservative path below the cutoff and reproduces the degree-2 versus degree-20 example without losing unknown input errors.

These steps are deliberately narrow. The existing reviews already contain extensive proposals for new scales, complex sectors, integration, uniform asymptotics, and general capability metadata. Repeating those would not add useful development guidance here. The immediate opportunity is to make the existing shared primitives and their user-visible policies more dependable before extending their reach.

8.3 Performance measurements that answer these particular questions

For N01, measure the number of enclosure evaluations, the sequence of enclosure orders, and the amount of interval progress reused. Compare a state-preserving internal controller with the external adapter, which may redo earlier work. Include relative requests at several root magnitudes; a unit-scale example alone is not enough to choose a universal digit heuristic.

For C07, measure distinct realness proof obligations and cache hits within an operation, using the expression together with its proof context as the key. The measurement should reveal whether adding a gate causes repeated expensive proofs for identical coefficients, rather than merely reporting total time for a large expansion.

For the closed-frontier implementation, measure strict-interior pair products, additional boundary pair products, boundary polynomial size, and the difference between the actual degree and the conservative bound. The extra boundary work may be tiny for sparse low-dimensional supports and substantial for a resonant layer with many contributions. A controlled fallback is part of the design, not an afterthought.

For N02, measure whether an already satisfied lower request causes source replay. The desired monotone-policy fast path should need no new coefficient generation. The existing inverse statistics already make the distinction between reused blocks and new work inspectable; use that evidence instead of treating a smaller printed expression as a faster computation by default.[9]

8.4 Kernel-specific behavior and regression ownership

The native comparison results should be retained with the exact kernel version and invocation. An unevaluated `AsymptoticSolve` result is a fact about a particular call, not a permanent property of a mathematical family. Conversely, the package’s own error-bound and realness tests should not rely on native `AsymptoticSolve` failing. Their correctness obligations follow from their represented function and remainder, independently of competing solver behavior.

For C04, the independent polynomial oracle is useful because it does not call the package’s own coefficient or residual helpers. For N01, the absolute and explicit-order controls isolate the precision planner. For C07, the acceptance matrix crosses public entry points. For N02, the tests express a deliberately chosen policy. Each regression has a different owner and a different success criterion; combining them into a single vague “asymptotic correctness” score would conceal those distinctions.

9 Conclusions

The package’s strongest role is a specialized generalized inverse calculus with retained structure and evidence, not a universal replacement for native asymptotics. The executed comparisons show ordinary overlap, a useful factored special-function representation, and two concrete generalized inverse invocations where the package returned explicit blocks while the tested native equation solver remained unevaluated. That is a more defensible comparison than claiming that Wolfram Language lacks logarithmic, fractional, or non-Taylor asymptotics.

The additional certificate finding is narrow but important: exact proof arithmetic and adaptive proof planning are separate responsibilities. The observed relative-accuracy failure does not invalidate the interval proof; it shows that a successful proof can remain too weak indefinitely unless the controller is allowed to increase arithmetic precision on that path. The lower-cutoff refinement behavior likewise deserves an explicit policy, not an exaggerated mathematical diagnosis.

The strengthened existing findings are more directly about the mathematical contract. A real interface should not accept a nonreal constant merely because it arrives through a different entry point. A nonlinear operation must not discard a known boundary logarithm and then claim the smaller input error class. Native witnesses now demonstrate both behaviors in the pinned snapshot, and the independent closed-frontier prototype makes one sharper correction executable.

The practical next step is therefore not another broad feature wishlist. It is a set of small, separately testable changes: repair the shared nonlinear boundary rule, align realness checks, make certificate

precision adaptation accuracy-aware, and decide what refinement promises. The attached artifacts provide reproductions and implementation starting points while keeping their validation status explicit.

A Archive contents and reproduction

The archive is self-contained for reading the article and running the Python prototype tests. Native reproductions require a Wolfram installation or evaluator and the audited package. No font files and no complete copy of the repository are included.

File or directory	Purpose and validation status
article/audit.tex, article/audit.pdf	This article and its compiled PDF.
evidence/native_observations.json	Structured transcriptions of successful individual native evaluations; not a raw complete session log.
evidence/novelty_ledger.json	Baseline sources, search terms, exclusions, and new-versus-strengthened classification.
evidence/python_test_log.txt	Actual stdout/stderr from the independent Python test run.
evidence/python_validation.json	Python/SymPy versions, test counts, and oracle seed.
evidence/patch_validation.json	Explicit validation limits for candidate fixes.
code/closed_frontier.py	Executed rational-weight sparse boundary prototype.
tests/test_closed_frontier.py	Executed tests, including 40 independent-oracle subcases.
code/reproduce.wl	Aggregate native reproduction runner, authored but not executed as one complete run.
tests/DesiredBehavior.wlt	Proposed regression expectations; intentionally not all satisfied by the audited snapshot. Complete TestReport not run.
code/AccuracyAwareCertificate.wl	Opt-in certificate adapter candidate; complete artifact not natively validated.
code/patch_certificate.py	Hash-checked generator of experimental patched copies; does not overwrite a checkout.

From the archive root, run the independent tests with:

```
python -m pip install -r requirements.txt
python -m unittest discover -s tests -p 'test_*.py' -v
```

The observed environment was Python 3.13.5 and SymPy 1.14.0. The supplied dependency file pins the tested SymPy version.

A native local reproduction can use an already downloaded audited standalone:

```
wolframscript -file code/reproduce.wl /path/to/AsymptoticInverse.wl
```

Without a package argument, the runner downloads the pinned public standalone. It writes a new `audit-native-results.json` in the working directory, recording the actual kernel, package hash, inputs, outputs, messages, and elapsed times. Those new results should not be confused with the already supplied observation transcriptions.

To generate experimental certificate copies from an exact checkout:

```
python code/patch_certificate.py /path/to/Asymptotic \
--output /path/outside/checkout/certificate-candidate
```

The generator rejects unexpected source identities. Applying such copies to a development checkout, rebuilding the standalone, and running the repository’s full native suite remain integration work, not completed validation reported by this article.

To rebuild the article:

```
cd article
latexmk -pdf -interaction=nonstopmode -halt-on-error audit.tex
```

A PowerShell caller can run the same final command after changing directory. The TeX source uses installed TeX packages and does not need downloaded proprietary fonts.

B Source-inspection map

This map identifies the main inspected implementation paths, not an assertion that every repository line or every test was reviewed. All links in the bibliography resolve to the pinned commit.

Source module(s)	Inspected responsibilities
<code>AsymptoticInverse.wl</code> in Kernel/	Public declarations; exactness and sparse jets; precision propagation; analytic forward helpers; endpoint normalization; residual/numerical dispatch and module loading.
<code>SeriesOperations.wl</code> , <code>SeriesArithmetic.wl</code>	Ordered representation, observable and nonlinear operations, scalar realness gate, arithmetic and refinement dispatch.
<code>InverseCertificates.wl</code>	Exact interval arithmetic, elementary enclosures, domain proof, residual certificate, precision planning, and adaptive loop.
<code>RefinementState.wl</code>	State signatures, prefix reuse, boundary calculation, inverse assembly, and lower-cutoff behavior.
<code>NumericalInverseChecks.wl</code>	Reference-root recovery, source-domain check, observable recovery, and numerical evidence labels.

Source module(s)	Inspected responsibilities
<code>GammaForward.wl</code> , <code>GammaInverseOperations.wl</code>	Positive product normalization, logarithmic carriers, and inverse-observable precision clipping.
<code>NativeSpecialFunctions.wl</code> , <code>ParameterizedSpecialFunctions.wl</code>	Structured native import, source realness, projection, fixed-parameter analytic/Frobenius composition.
<code>FlatSectorOperations.wl</code> , <code>FourierCoefficients.wl</code>	Separate sector errors, convolution, exact frequencies, conjugacy, inverse coefficients, and formal residuals.
<code>ReciprocalLogOperations.wl</code> , <code>SourceCoordinates.wl</code>	Specialized composition/differentiation and exact/nonexact source-chart reconstruction.
<code>SeriesEnvelopeArithmetic.wl</code> , <code>InverseFunctionSyntax.wl</code>	Composite approach recovery and envelopes; scoped inverse-call and condition parsing.

The review register, public guide, README, review index, original Review 2 finding, and selected original review sections were also examined. Repository-wide native success, exhaustive branch coverage, and general proof of all supported remainder contracts are not claimed.

C Evidence interpretation

The main text uses “observed” only for a successful native evaluator result or an actual local Python execution. Source-traced explanations are identified by the relevant implementation module. Proposed changes are not relabeled as validated fixes merely because their anchors match inspected code or their mathematical intent is clear.

The attached native record preserves useful numerical summaries of rational certificates rather than printing every enormous rational endpoint in the article. Its decimal error values are display approximations to the recorded certificate bounds, not the arithmetic used to establish them. The complete reproduction runner can produce fresh results in a local kernel for exact inspection.

A final distinction is important for the two strengthened old findings. This audit does not add a new proof of Review 2’s already correct nonlinear counterexample, nor does it claim that the closed-boundary idea first appears here. It adds execution against the current standalone and an independently tested implementation. For C07, it replaces an abstract validation concern with exact public inputs and divergent native outcomes. Those are the specific reasons the two subjects remain in a report that otherwise excludes the existing backlog.

References

- [1] Vladimir Reshetnikov, *Asymptotic repository README*, audited commit 921387e5ba12. [README.md](#).
- [2] *Review index*. [code-review/README.md](#). All repository references below use the same pinned commit.
- [3] *Consolidated code-review status*. [docs/development/CODE_REVIEW_STATUS.md](#).
- [4] *AsymptoticInverse user guide*. [AsymptoticInverse/Documentation/UserGuide.md](#). In particular the input-remainder declaration and public operation contracts.

- [5] *Core package*. [AsymptoticInverse/Kernel/AsymptoticInverse.wl](#). In particular pConst, pMul, unitSeriesPrecision, pUnitSeries, fwd, and fwdAnalytic; inspected precision/forward region approximately lines 258–500.
- [6] *Ordered series calculus*. [AsymptoticInverse/Kernel/SeriesOperations.wl](#). Representation, observable, nonlinear, and refinement paths.
- [7] *Arithmetic dispatch*. [AsymptoticInverse/Kernel/SeriesArithmetic.wl](#). In particular seriesRegularOperand and public operation dispatch.
- [8] *Exact inverse certificates*. [AsymptoticInverse/Kernel/InverseCertificates.wl](#). certAttempt and InverseCertificate; automatic order selection and adaptive loop in the constructor beginning near line 293.
- [9] *Retained inverse state*. [AsymptoticInverse/Kernel/RefinementState.wl](#). refineStoredInverse, beginning near line 120.
- [10] *Numerical inverse evidence*. [AsymptoticInverse/Kernel/NumericalInverseChecks.wl](#).
- [11] *Gamma forward normalization*. [AsymptoticInverse/Kernel/GammaForward.wl](#).
- [12] *Gamma/Barnes inverse observable operations*. [AsymptoticInverse/Kernel/GammaInverseOperations.wl](#).
- [13] *Native special-function adapter*. [AsymptoticInverse/Kernel/NativeSpecialFunctions.wl](#). In particular structured tree import, real projection, carrier separation, and exactness checks.
- [14] *Fixed-parameter special-function composition*. [AsymptoticInverse/Kernel/ParameterizedSpecialFunctions.wl](#).
- [15] *Finite flat-sector operations*. [AsymptoticInverse/Kernel/FlatSectorOperations.wl](#).
- [16] *Fourier-polynomial coefficient algebra*. [AsymptoticInverse/Kernel/FourierCoefficients.wl](#).
- [17] *Composite error envelopes*. [AsymptoticInverse/Kernel/SeriesEnvelopeArithmetic.wl](#).
- [18] *Exact source coordinates*. [AsymptoticInverse/Kernel/SourceCoordinates.wl](#).
- [19] *Existing code review 2*, especially finding F01 and its proposed conservative repair. [code-review/code-review-2/article/asymptotic-review.tex](#), approximately lines 256–324.
- [20] Wolfram Research, *Series*, Wolfram Language documentation, accessed 9 September 2026. <https://reference.wolfram.com/language/ref/Series.html>.
- [21] Wolfram Research, *SeriesData*, documentation, accessed 9 September 2026. <https://reference.wolfram.com/language/ref/SeriesData.html>.
- [22] Wolfram Research, *InverseSeries*, documentation, accessed 9 September 2026. <https://reference.wolfram.com/language/ref/InverseSeries.html>.
- [23] Wolfram Research, *ComposeSeries*, documentation, accessed 9 September 2026. <https://reference.wolfram.com/language/ref/ComposeSeries.html>.
- [24] Wolfram Research, *AsymptoticSolve*, documentation, accessed 9 September 2026. <https://reference.wolfram.com/language/ref/AsymptoticSolve.html>.
- [25] Wolfram Research, *Asymptotic*, documentation, accessed 9 September 2026. <https://reference.wolfram.com/language/ref/Asymptotic.html>.
- [26] *Native observation record accompanying this audit*. Archive file evidence/native_observations.json. Structured manual transcriptions of successful Wolfram 15.0.0 evaluator responses; inputs, controls, and provenance included.
- [27] *Executed independent prototype tests*. Archive files evidence/python_test_log.txt, evidence/python_validation.json, and tests/test_closed_frontier.py. Thirteen test methods, including forty deterministic randomized oracle subcases.